

Authentication

JWT · Sessions · Custom Auth · NextAuth v5 · Clerk · OAuth · RBAC · Security

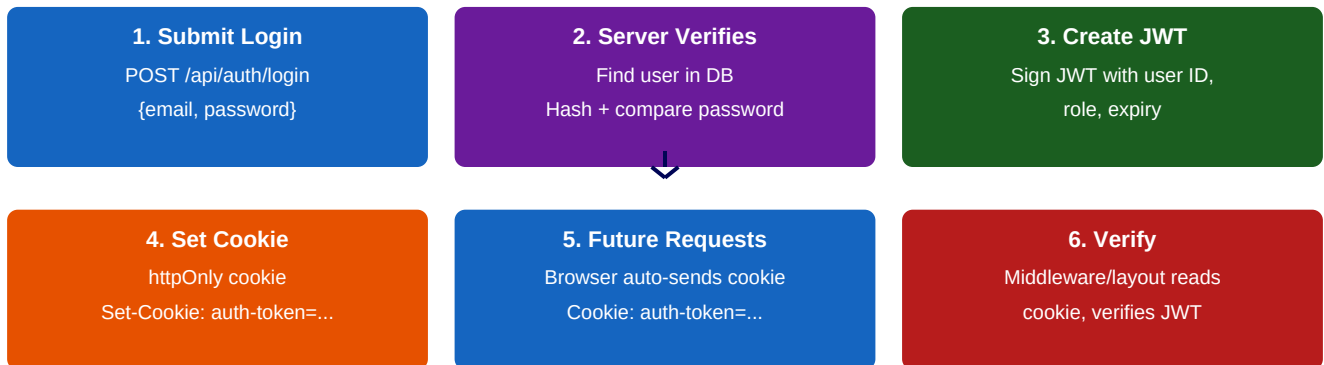
Akshay Chavhan | Target: Syngenta India + All Companies | April 2026

■ BEGINNER LEVEL

Authentication vs Authorization

	Authentication	Authorization
Question	WHO are you?	WHAT can you do?
Example	Login with email/password	Can Akshay access /admin?
Happens	First (proves identity)	Second (checks permission)
Result	"Yes, you are Akshay"	"Akshay is admin — allowed"
Real world	Hotel check-in	Room key access level
Next.js	Login form, OAuth	Role checks, route guards

The Complete Auth Flow



What is JWT?

JWT (JSON Web Token) is a signed token storing user identity. It has 3 parts: Header (algorithm), Payload (user data), Signature (tamper-proof). NOT encrypted — anyone can decode payload. IS signed — server verifies authenticity. Only store user ID, role, expiry — NEVER passwords or sensitive data.

HEADER	PAYLOAD	SIGNATURE
eyJhbGciOiJIUzI1NiJ9...	eyJzdWIiOiJ1c2VyXzEy...	SfLKxwRJSMeKKF2QT4fw...
{"alg": "HS256", "typ": "JWT"}	{"sub": "user_123", "role": "admin"}	HMACSHA256(header+payload, secret)

■ JWT is like a government ID card: Anyone can READ it (not encrypted) but nobody can FAKE it (signed).
Server checks the signature — not the content.

Three Ways to Build Auth in Next.js

Option	What it is	Setup Time	Cost	Best For
Custom	Build everything yourself	Days/weeks	Dev time	Enterprise, full control
NextAuth v5	Open source library	Hours	Free	Mid-size, open source
Clerk	Managed service	30 minutes	Free up to 10k MAU	Startups, fast build

■ INTERMEDIATE LEVEL

JWT Strategy vs Session Strategy

	JWT Strategy	Database Session Strategy
How it works	User data inside token itself	Random ID → DB lookup
DB call per request	No (~1ms verify)	Yes (5-50ms DB lookup)
Can be revoked?	No — valid until expiry	Yes — delete DB record instantly
Edge Runtime	Yes (stateless)	No (needs DB)
Scales	Infinitely (no DB)	Needs DB infrastructure
Sign out everywhere	Hard	Easy — delete all sessions
Use when	Performance, scale, edge	High security, instant revoke

Custom Auth — JWT with Credentials

```

// app/actions/auth.ts
'use server';
import bcrypt      from 'bcryptjs';
import { SignJWT } from 'jose';
import { cookies } from 'next/headers';
import { redirect } from 'next/navigation';

const SECRET = new TextEncoder().encode(process.env.JWT_SECRET!);

export async function login(formData: FormData) {
  const email    = formData.get('email') as string;
  const password = formData.get('password') as string;

  // Step 1: Find user
  const user = await prisma.user.findUnique({ where: { email } });
  if (!user) return { error: 'Invalid credentials' };

  // Step 2: Verify password hash
  const valid = await bcrypt.compare(password, user.passwordHash);
  if (!valid) return { error: 'Invalid credentials' };

  // Step 3: Create JWT
  const token = await new SignJWT({
    sub: user.id, role: user.role, email: user.email
  })
    .setProtectedHeader({ alg: 'HS256' })
    .setExpirationTime('7d')
    .sign(SECRET);

  // Step 4: Set httpOnly cookie
  const cookieStore = cookies();
  cookieStore.set('auth-token', token, {
    httpOnly: true, secure: true, sameSite: "lax",
    maxAge: 60 * 60 * 24 * 7
  });

  redirect('/dashboard');
}

```

Get Current User in Server Components

```
// lib/auth.ts
import { jwtVerify } from 'jose';
import { cookies } from 'next/headers';

const SECRET = new TextEncoder().encode(process.env.JWT_SECRET!);

export async function getCurrentUser() {
  const cookieStore = cookies();
  const token = cookieStore.get('auth-token')?.value;
  if (!token) return null;

  try {
    const { payload } = await jwtVerify(token, SECRET);
    return {
      id: payload.sub as string,
      role: payload.role as string,
      email: payload.email as string,
    };
  } catch {
    return null; // token expired or invalid
  }
}

// Usage in Server Component:
import { getCurrentUser } from '@lib/auth';
import { redirect } from 'next/navigation';

export default async function DashboardPage() {
  const user = await getCurrentUser();
  if (!user) redirect('/login');
  return <h1>Welcome {user.email}</h1>;
}
```

NextAuth v5 — Setup

```

// auth.ts (root level)
import NextAuth from 'next-auth';
import Google from 'next-auth/providers/google';
import Credentials from 'next-auth/providers/credentials';

export const { handlers, auth, signIn, signOut } = NextAuth({
  providers: [
    Google({
      clientId: process.env.GOOGLE_CLIENT_ID!,
      clientSecret: process.env.GOOGLE_CLIENT_SECRET!,
    }),
    Credentials({
      async authorize(credentials) {
        const { email, password } = credentials as any;
        const user = await prisma.user.findUnique({ where: { email } });
        if (!user) return null;
        const valid = await bcrypt.compare(password, user.passwordHash);
        if (!valid) return null;
        return { id: user.id, email: user.email, role: user.role };
      },
    }),
  ],
  callbacks: {
    jwt({ token, user }) {
      if (user) token.role = (user as any).role;
      return token;
    },
    session({ session, token }) {
      session.user.role = token.role as string;
      return session;
    },
  },
  pages: { signIn: '/login' },
});

// app/api/auth/[...nextauth]/route.ts
import { handlers } from '@auth';
export const { GET, POST } = handlers;

// middleware.ts
export { auth as middleware } from '@auth';
export const config = { matcher: ['/dashboard/:path*'] };

```

Clerk — Fastest Setup

```
// middleware.ts – one import does everything
import { clerkMiddleware, createRouteMatcher } from '@clerk/nextjs/server';

const isProtected = createRouteMatcher(['/dashboard(.*)', '/admin(.*)']);

export default clerkMiddleware(async (auth, req) => {
  if (isProtected(req)) await auth.protect();
});

export const config = { matcher: ['/(?!_next/static|_next/image).*'] };

// app/layout.tsx
import { ClerkProvider } from '@clerk/nextjs';
export default function RootLayout({ children }) {
  return <ClerkProvider><html><body>{children}</body></html></ClerkProvider>;
}

// Login page – pre-built UI, zero code!
import { SignIn } from '@clerk/nextjs';
export default function LoginPage() { return <SignIn />; }

// Server Component – get user
import { auth, currentUser } from '@clerk/nextjs/server';
export default async function Dashboard() {
  const { userId } = await auth();
  if (!userId) redirect('/login');
  const user = await currentUser();
  return <h1>Welcome {user?.firstName}</h1>;
}

// Client Component
'use client';
import { useUser, UserButton } from '@clerk/nextjs';
export function Header() {
  const { user } = useUser();
  return <div>{user?.emailAddresses[0].emailAddress} <UserButton /></div>;
}
```

■ EXPERT LEVEL

Defence in Depth — Never One Layer Only

CVE-2025-29927 proved middleware alone can be bypassed. Always check auth at **multiple layers**. Each layer is a safety net.

```
// LAYER 1: Middleware (edge – blocks before Lambda runs)
export default clerkMiddleware(async (auth, req) => {
  if (isProtected(req)) await auth.protect();
});

// LAYER 2: Layout (server – catches middleware bypass)
// app/dashboard/layout.tsx
export default async function DashboardLayout({ children }) {
  const { userId } = await auth();
  if (!userId) redirect('/login'); // double check
  return <{children}>;
}

// LAYER 3: Page (verify specific role)
// app/admin/page.tsx
export default async function AdminPage() {
  const { userId } = await auth();
  const user = await prisma.user.findUnique({ where: { id: userId! } });
  if (user?.role !== 'admin') redirect('/unauthorized');
  return <AdminDashboard />;
}

// LAYER 4: Server Action (MOST critical – always check!)
'use server';
export async function deleteUser(targetId: string) {
  const { userId } = await auth();
  if (!userId) throw new Error('Unauthorized');
  const me = await prisma.user.findUnique({ where: { id: userId } });
  if (me?.role !== 'admin') throw new Error('Forbidden');
  await prisma.user.delete({ where: { id: targetId } });
}
```

Password Hashing — Never Store Plain Text

```
import bcrypt from 'bcryptjs';

// REGISTER – always hash before saving
export async function register(email: string, password: string) {
  const hash = await bcrypt.hash(password, 12); // 12 salt rounds
  await prisma.user.create({
    data: { email, passwordHash: hash } // NEVER store plain password
  });
}

// LOGIN – compare with hash
export async function verifyPassword(
  input: string, stored: string
): Promise<boolean> {
  return bcrypt.compare(input, stored); // true or false
}

// Even if database is stolen:
// password: "secret123"
// hash:      "$2a$12$K8IH3x9..." ← mathematically impossible to reverse
```

OAuth Flow — How Google Login Works

```
User clicks "Login with Google"
↓
Your app redirects to Google:
  https://accounts.google.com/oauth?client_id=YOUR_ID&scope=email
↓
User logs in on Google
↓
Google redirects back:
  yoursite.com/api/auth/callback/google?code=abc123
↓
Your server exchanges code for tokens (server-to-server):
  POST https://oauth2.googleapis.com/token { code, client_id, secret }
↓
Google returns: { access_token, id_token }
↓
Decode id_token → get { email, name, picture, sub }
↓
Find or create user in YOUR database
↓
Create JWT or session → set httpOnly cookie → redirect to dashboard

// NextAuth / Clerk handles ALL of the above automatically!
```

NextAuth vs Clerk vs Custom — Full Comparison

Feature	Custom	NextAuth v5	Clerk
Setup time	Days/weeks	Hours	30 min
Cost	Dev time only	Free forever	Free up to 10k MAU
Data ownership	Full	Full	Clerk hosts users
OAuth providers	Build each	80+ built-in	30+ built-in
Pre-built UI	Build it	Build it	SignIn/SignUp ready
MFA support	Build it	Limited	Built-in
Edge Runtime	Yes (jose)	Yes (JWT)	Yes (optimized)
RBAC	Build it	Via callbacks	Built-in
Best for	Enterprise	Mid-size OSS	Startups, fast ship

Security Best Practices

Practice	Wrong	Right
Token storage	localStorage (XSS risk!)	httpOnly cookie
Passwords	Store plain text	bcrypt.hash(pwd, 12)
JWT payload	Store PII, passwords	Only ID, role, expiry
Auth layers	Middleware only	Middleware + layout + action
JWT expiry	Never expires	Short-lived (7d or less)

Cookie flags	No flags	httpOnly + Secure + SameSite
--------------	----------	------------------------------

■ Critical Gotchas

■ Gotcha 1: Never use localStorage for auth tokens. JavaScript can always read localStorage — an XSS attack can steal your token. Always use httpOnly cookies. Server sets them, JS cannot read them, they auto-send with every request.

■ Gotcha 2: Never store passwords in plain text. Always hash with bcrypt before saving. Cost factor 12+ is recommended. Even if your database is stolen, bcrypt hashes are computationally infeasible to reverse.

■ Gotcha 3: JWT tokens cannot be revoked. Deleting the cookie logs the user out on the client, but if an attacker captured the JWT it is still valid until expiry. Solutions: very short expiry (Clerk uses 60s), JWT denylist in Redis, or use database sessions.

■ Gotcha 4: Do NOT rely on middleware alone for auth (CVE-2025-29927). Always verify auth in Server Components, layouts, Server Actions, and API routes too. Middleware is the first line, not the only line.

■ Best Practice: Choose Clerk for new projects that need fast development. Choose NextAuth for free/open-source with 80+ OAuth providers. Choose custom only when you need complete control or specific enterprise compliance requirements.

■ Best Practice: Get user identity from the SERVER (auth() or cookies()) — never from client-provided parameters. Never do getUser(clerkId) where clerkId comes from the client — that's a security vulnerability.

■ INTERVIEW Q&A WITH DETAILED ANSWERS

Beginner Level

BEGINNER

Q1. What is the difference between authentication and authorization?

Answer: Authentication = WHO are you? Proving identity — login with email/password or Google. Result: "Yes, you are Akshay." Authorization = WHAT can you do? Checking permissions — can Akshay access /admin? Result: "Yes, Akshay is admin — allowed." Authentication always happens first. Hotel analogy: check-in = authentication, room key level = authorization.

BEGINNER

Q2. What is a JWT and what should you store inside it?

Answer: JWT (JSON Web Token) is a signed token with 3 parts: Header (algorithm), Payload (user data), Signature (tamper-proof). NOT encrypted — anyone can decode the payload. IS signed — server verifies it was not modified. Store only: user ID, role, expiry timestamp. Never store: passwords, PII, credit card data, sensitive business data.

BEGINNER**Q3. Where should you store JWT tokens in Next.js? Why not localStorage?**

Answer: Always in an httpOnly cookie — never localStorage. localStorage: JavaScript can always read it — XSS attacks steal tokens instantly. httpOnly cookie: JavaScript cannot read it at all — even if attacker injects JS, cookie is safe. Also add Secure flag (HTTPS only) and SameSite: lax (prevents CSRF). httpOnly cookie auto-sends with every request so middleware and Server Components read it.

Intermediate Level

INTERMEDIATE**Q4. What is the difference between JWT and database session strategies?**

Answer: JWT: all user data inside token, no DB lookup per request (~1ms), stateless, scales infinitely, works on Edge Runtime, but cannot be revoked before expiry. Database sessions: random session ID stored in DB, requires DB lookup per request (5-50ms), instant revocation by deleting DB record, does not work on Edge Runtime. Use JWT for performance and scale. Use DB sessions for high security and instant sign-out-everywhere.

INTERMEDIATE**Q5. How do you implement Google OAuth login with NextAuth?**

Answer: Create auth.ts at project root, configure NextAuth with Google provider passing clientId and clientSecret from env variables. Export handlers and create app/api/auth/[...nextauth]/route.ts exporting GET and POST. Use callbacks to add role to JWT and session. Export auth as middleware for route protection. In Server Components, call auth() to get session. NextAuth handles the entire OAuth code exchange flow automatically.

INTERMEDIATE**Q6. What is the simplest way to add authentication to a Next.js app?**

Answer: Clerk. Install @clerk/nextjs, wrap RootLayout with ClerkProvider, export clerkMiddleware from middleware.ts with your protected routes, create login page with just the SignIn component from Clerk. Done in 30 minutes. Clerk handles: OAuth providers, session management, JWT refresh, pre-built UI, MFA, RBAC — all out of the box.

INTERMEDIATE**Q7. How do you pass role to JWT in NextAuth?**

Answer: In the callbacks section: in the jwt callback, check if user object exists (first sign-in), copy user.role to token.role. In the session callback, copy token.role to session.user.role. Now session.user.role is available in Server Components via auth(). Extend the Session and JWT TypeScript types to avoid type errors.

Expert / Syngenta-Style Questions

EXPERT**Q8. What is defence in depth in auth and why is it important?**

Answer: Never rely on one auth check alone. CVE-2025-29927 showed middleware can be bypassed. Layer 1 (Middleware): blocks unauthenticated users at the edge. Layer 2 (Layout): Server Component catches anything middleware missed. Layer 3 (Page): verifies specific role for sensitive pages. Layer 4 (Server Action): ALWAYS verify before any mutation. Layer 5 (API Route): verify on every protected endpoint. Multiple layers = attacker must bypass all.

EXPERT**Q9. Why can JWT tokens not be revoked and what are the solutions?**

Answer: JWTs are stateless — server has no record of issued tokens. Once created, valid until expiry regardless of logout. Solutions: (1) Very short expiry — Clerk uses 60 seconds, auto-refreshed, minimizes damage window. (2) JWT denylist in Redis — on logout, add token ID to Redis blacklist, check on each request. (3) Database sessions — immediate revocation by deleting session record. (4) Rotate tokens with refresh token pattern — short access + long refresh.

SYNGENTA**Q10. Design the complete authentication system for Syngenta farmer portal.**

Answer: Use Clerk for pre-built components and fast development. ClerkProvider in root layout. clerkMiddleware protecting /dashboard/*, /admin/*, /portal/*. Three roles in Clerk publicMetadata: admin (Syngenta staff), manager (regional), farmer (end user). Custom /login and /register pages with Clerk SignIn/SignUp components. Server Components: auth() + currentUser() — never trust client-provided IDs. Server Actions: always call auth() first, check role before mutation. Rate limiting on /api/auth/* with Upstash to prevent brute force. All cookies managed by Clerk with httpOnly + Secure + SameSite automatically.

EXPERT**Q11. A developer passed clerkId as a function parameter from the client. What is the security issue?**

Answer: The client controls what clerkId value it sends — any user can pass any ID and read another user's data. This is a broken object level authorization (BOLA/IDOR) vulnerability. Fix: always get clerkId from auth() on the server — it reads from the verified JWT cookie, which only the server can verify. Remove clerkId from function parameters entirely. The server session is the only source of truth for user identity.